K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 04 TASKS

TASK 01:

```
#include <iostream>
using namespace std;

class Node
{
public:
  int data;
  Node *next;

  Node(int val)
  {
    data = val;
    next = nullptr;
  }
};

class SLL
{
  Node *head;
  Node *tail;

public:
  SLL()
  {
    head = tail = nullptr;
  }

  void insertFront(int val)
  {
    Node *newNode = new Node(val);

    if (head == nullptr)
    {
      head = newNode;
      tail = newNode;
      return;
    }

    newNode->next = head;
    head = newNode;
  }
```

DSA LAB 04 TASKS

```
void insertBack(int val)
{
   Node *newNode = new Node(val);

   if (head == nullptr)
   {
      head = newNode;
      tail = newNode;

      return;
   }

   tail->next = newNode;
   tail = newNode;
}

void insertPos(int pos, int val)
{
   if (pos == 1)
   {
      insertFront(val);
      return;
   }

   Node *newNode = new Node(val);

   Node *temp;
   temp = head;

   for (int i = 1; i < pos - 1; i++)
   {
      if (temp == nullptr)
      {
         cout << "\nInvalid position. ";
         delete newNode;
         return;
      }
      temp = temp->next;
   }

   if (temp == nullptr)
```

DSA LAB 04 TASKS

```
    {
        cout << "\nInvalid position. ";
        delete newNode;
        return;
    }

    newNode->next = temp->next;
    temp->next = newNode;

    if (newNode->next == nullptr)
    {
        tail = newNode;
    }
}

void removeFront()
{
    if (head == nullptr)
    {
        return;
    }

    Node *temp;
    temp = head;
    head = head->next;
    delete temp;

    if (head == nullptr)
    {
        tail = nullptr;
    }
}

void removeBack()
{
    if (head == nullptr)
    {
        return;
    }

    if (head->next == nullptr)
    {
```

DSA LAB 04 TASKS

```
        delete head;
        head = nullptr;
        return;
    }

    Node *temp = head;
    while (temp->next != tail)
    {
        temp = temp->next;
    }

    delete temp->next;
    temp->next = nullptr;
    tail = temp;
}

void deletePos(int pos)
{
    if (head == nullptr)
    {
        cout << "\nList is empty. ";
        return;
    }

    if (pos == 1)
    {
        removeFront();
        return;
    }

    Node *temp;
    temp = head;

    for (int i = 1; i < pos - 1; i++)
    {
        if (temp->next == nullptr)
        {
            cout << "\nInvalid position. ";
            return;
        }

        temp = temp->next;
```

DSA LAB 04 TASKS

```
    }

    Node *deletenode = temp->next;

    if (deletenode == nullptr)
    {
        cout << "\nInvalid position.";
        return;
    }

    if (deletenode == tail)
    {
        tail = temp;
    }

    temp->next = deletenode->next;

    delete deletenode;
}

void display()
{
    Node *temp = head;

    cout << "\nList: ";

    while (temp != nullptr)
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }

    cout << "NULL";

    if (head != nullptr)
    {
        cout << "\nHead = " << head->data;
        cout << "\nTail = " << tail->data;
    }
    else
    {
        cout << "\nHead = NULL";
```

DSA LAB 04 TASKS

```
        cout << "\nTail = NULL";
    }

    cout << endl;
  }
};

int main()
{
  SLL list;
  int n;

  cout << "\nEnter number of values to be stored in linked list: ";
  cin >> n;

  for (int i = 1; i <= n; i++)
  {
    int val, choice;

    cout << "\nEnter value " << i << ": ";
    cin >> val;

    cout << "1. Insert at Front";
    cout << "\n2. Insert at Back";
    cout << "\nEnter choice: ";
    cin >> choice;

    if (choice == 1)
    {
      list.insertFront(val);
    }
    else if (choice == 2)
    {
      list.insertBack(val);
    }
    else
    {
      cout << "Invalid choice. Value not inserted.\n";
      i--;
    }
  }
```

DSA LAB 04 TASKS

```
cout << "\n\n===== INITIAL LIST =====";
list.display();

int choice;

do
{
  cout << "\n\n===== MENU =====";
  cout << "\n1. Remove Front";
  cout << "\n2. Remove Back";
  cout << "\n3. Insert at Position";
  cout << "\n4. Delete at Position";
  cout << "\n5. Display List";
  cout << "\n0. Exit";

  cout << "\n\nEnter choice: ";
  cin >> choice;

  if (choice == 1)
  {
    list.removeFront();
    cout << "\nAfter removing front:";
    list.display();
  }

  else if (choice == 2)
  {
    list.removeBack();
    cout << "\nAfter removing back:";
    list.display();
  }

  else if (choice == 3)
  {
    int pos, val;

    cout << "\nEnter position: ";
    cin >> pos;

    cout << "Enter value: ";
    cin >> val;
```

DSA LAB 04 TASKS

```
        list.insertPos(pos, val);

        cout << "\nAfter insertion:";
        list.display();
    }

    else if (choice == 4)
    {
        int pos;

        cout << "\nEnter position to delete: ";
        cin >> pos;

        list.deletePos(pos);

        cout << "\nAfter deletion:";
        list.display();
    }

    else if (choice == 5)
    {
        list.display();
    }

    else if (choice == 0)
    {
        cout << "\nProgram ended.";
    }

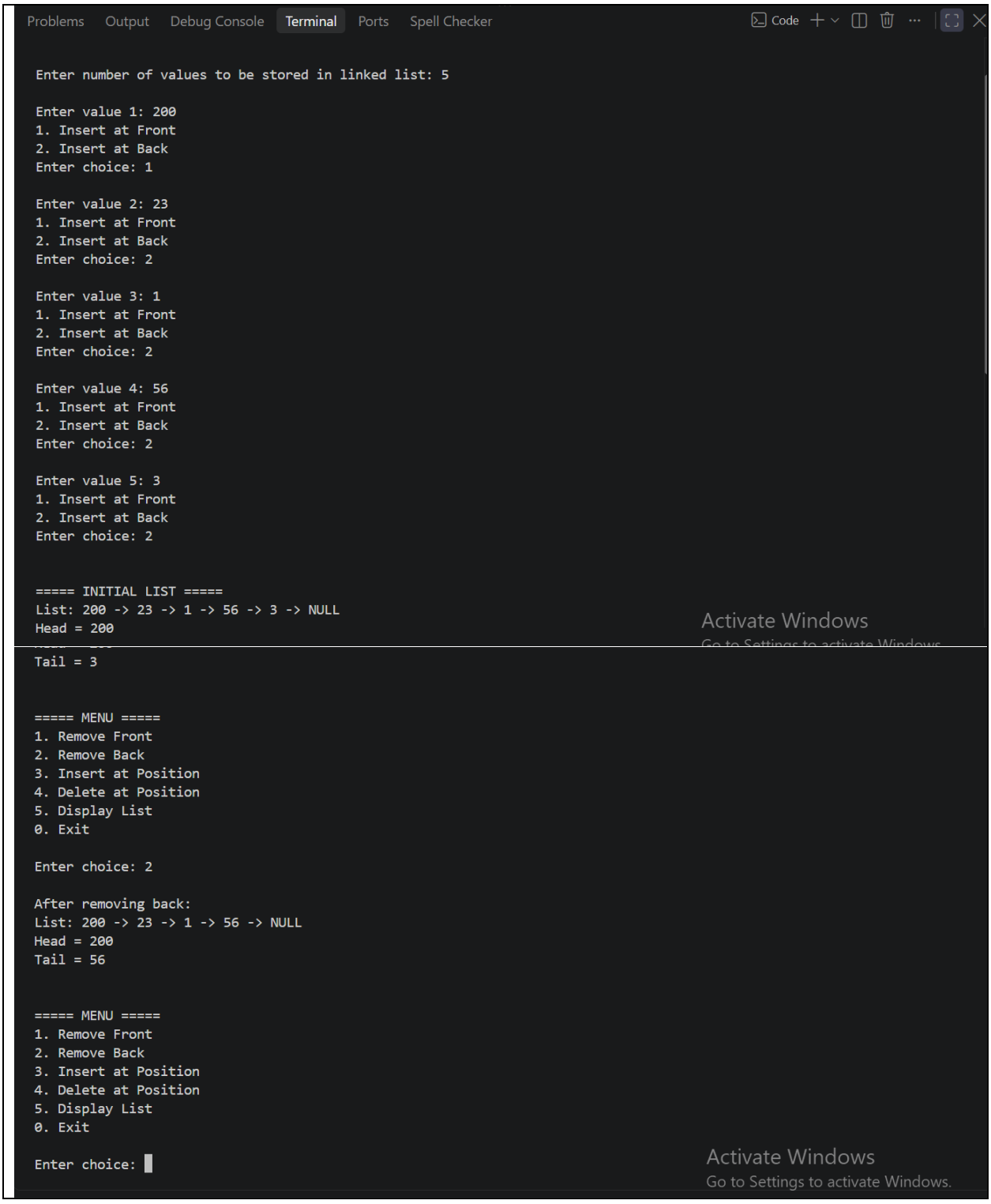
    else
    {
        cout << "\nInvalid choice.";
    }

  } while (choice != 0);

  return 0;
}
```

K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 04 TASKS



TASK 02:

K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 04 TASKS

```
#include <iostream>
using namespace std;

class Node
{
public:
   int data;
   Node *next;

   Node(int val)
   {
      data = val;
      next = nullptr;
   }
};

class SLL
{
   Node *head;
   Node *tail;

public:
   SLL()
   {
      head = tail = nullptr;
   }

   void insertFront(int val)
   {
      Node *newNode = new Node(val);

      if (head == nullptr)
      {
         head = newNode;
         tail = newNode;
         return;
      }

      newNode->next = head;
      head = newNode;
   }
```

DSA LAB 04 TASKS

```
void insertBack(int val)
{
   Node *newNode = new Node(val);

   if (head == nullptr)
   {
      head = newNode;
      tail = newNode;

      return;
   }

   tail->next = newNode;
   tail = newNode;
}

void insertPos(int pos, int val)
{
   if (pos == 1)
   {
      insertFront(val);
      return;
   }

   Node *newNode = new Node(val);

   Node *temp;
   temp = head;

   for (int i = 1; i < pos - 1; i++)
   {
      if (temp == nullptr)
      {
         cout << "\nInvalid position. ";
         delete newNode;
         return;
      }
      temp = temp->next;
   }

   if (temp == nullptr)
   {
```

DSA LAB 04 TASKS

```
        cout << "\nInvalid position. ";
        delete newNode;
        return;
    }

    newNode->next = temp->next;
    temp->next = newNode;

    if (newNode->next == nullptr)
    {
        tail = newNode;
    }
}

void removeFront()
{
    if (head == nullptr)
    {
        return;
    }

    Node *temp;
    temp = head;
    head = head->next;
    delete temp;

    if (head == nullptr)
    {
        tail = nullptr;
    }
}

void removeBack()
{
    if (head == nullptr)
    {
        return;
    }

    if (head->next == nullptr)
    {
        delete head;
```

DSA LAB 04 TASKS

```
        head = nullptr;
        return;
    }

    Node *temp = head;
    while (temp->next != tail)
    {
        temp = temp->next;
    }

    delete temp->next;
    temp->next = nullptr;
    tail = temp;
}

void deletePos(int pos)
{
    if (head == nullptr)
    {
        cout << "\nList is empty. ";
        return;
    }

    if (pos == 1)
    {
        removeFront();
        return;
    }

    Node *temp;
    temp = head;

    for (int i = 1; i < pos - 1; i++)
    {
        if (temp->next == nullptr)
        {
            cout << "\nInvalid position. ";
            return;
        }

        temp = temp->next;
    }
```

DSA LAB 04 TASKS

```
    Node *deletenode = temp->next;

    if (deletenode == nullptr)
    {
        cout << "\nInvalid position.";
        return;
    }

    if (deletenode == tail)
    {
        tail = temp;
    }

    temp->next = deletenode->next;

    delete deletenode;
}

void display()
{
    Node *temp = head;

    cout << "\nList: ";

    while (temp != nullptr)
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }

    cout << "NULL";

    if (head != nullptr)
    {
        cout << "\nHead = " << head->data;
        cout << "\nTail = " << tail->data;
    }
    else
    {
        cout << "\nHead = NULL";
        cout << "\nTail = NULL";
```

DSA LAB 04 TASKS

```
    }

    cout << endl;
  }

  void removeDuplicates()
  {
    Node *temp;
    temp = head;

    while (temp != nullptr && temp->next != nullptr)
    {
      Node *nodeToDel;
      if (temp->data == temp->next->data)
      {
        nodeToDel = temp->next;
        temp->next = nodeToDel->next;

        if (nodeToDel == tail)
        {
          tail = temp;
        }

        delete nodeToDel;
      }
      else
      {
        temp = temp->next;
      }
    }
  }
};

int main()
{
  SLL list;
  int n;

  cout << "\nEnter number of values to be stored in linked list: ";
  cin >> n;

  for (int i = 1; i <= n; i++)
```

DSA LAB 04 TASKS

```
{
   int val, choice;

   cout << "\nEnter value " << i << ": ";
   cin >> val;

   cout << "1. Insert at Front";
   cout << "\n2. Insert at Back";
   cout << "\nEnter choice: ";
   cin >> choice;

   if (choice == 1)
   {
      list.insertFront(val);
   }
   else if (choice == 2)
   {
      list.insertBack(val);
   }
   else
   {
      cout << "Invalid choice. Value not inserted.\n";
      i--;
   }
}

cout << "\n\n===== INITIAL LIST =====";
list.display();

int choice;

do
{
   cout << "\n\n===== MENU =====";
   cout << "\n1. Remove Front";
   cout << "\n2. Remove Back";
   cout << "\n3. Insert at Position";
   cout << "\n4. Delete at Position";
   cout << "\n5. Display List";
   cout << "\n6. Display List after removing duplicates";
   cout << "\n0. Exit";
```

DSA LAB 04 TASKS

```
    cout << "\n\nEnter choice: ";
    cin >> choice;

    if (choice == 1)
    {
       list.removeFront();
       cout << "\nAfter removing front:";
       list.display();
    }

    else if (choice == 2)
    {
       list.removeBack();
       cout << "\nAfter removing back:";
       list.display();
    }

    else if (choice == 3)
    {
       int pos, val;

       cout << "\nEnter position: ";
       cin >> pos;

       cout << "Enter value: ";
       cin >> val;

       list.insertPos(pos, val);

       cout << "\nAfter insertion:";
       list.display();
    }

    else if (choice == 4)
    {
       int pos;

       cout << "\nEnter position to delete: ";
       cin >> pos;

       list.deletePos(pos);
```

DSA LAB 04 TASKS

```
        cout << "\nAfter deletion:";
        list.display();
    }

    else if (choice == 5)
    {
        list.display();
    }

    else if(choice==6)
    {
        cout<<"\nLIST AFTER REMOVING DUPLICATES(If Any): ";
        list.removeDuplicates();
        list.display();
    }

    else if (choice == 0)
    {
        cout << "\nProgram ended.";
    }

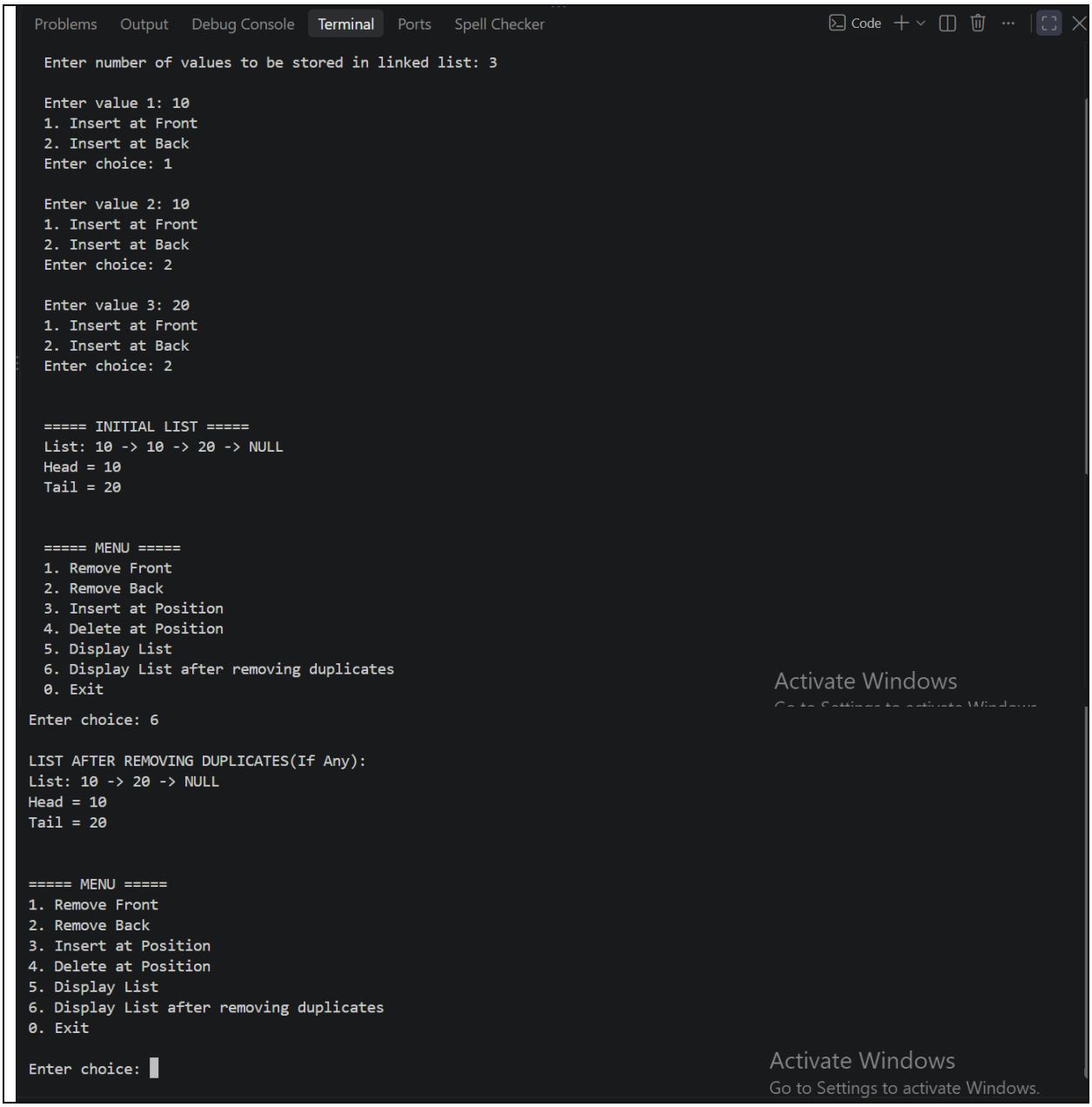
    else
    {
        cout << "\nInvalid choice.";
    }

  } while (choice != 0);

  return 0;
}
```

K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 04 TASKS



TASK 03:

```
#include <iostream>
using namespace std;

class Node
{
public:
   int data;
```

DSA LAB 04 TASKS

```
    Node *next;

    Node(int val)
    {
        data = val;
        next = nullptr;
    }
};

class SLL
{
    Node *head;
    Node *tail;

public:
    SLL()
    {
        head = tail = nullptr;
    }

    void insertFront(int val)
    {
        Node *newNode = new Node(val);

        if (head == nullptr)
        {
            head = newNode;
            tail = newNode;
            return;
        }

        newNode->next = head;
        head = newNode;
    }

    void insertBack(int val)
    {
        Node *newNode = new Node(val);

        if (head == nullptr)
        {
            head = newNode;
```

DSA LAB 04 TASKS

```
        tail = newNode;

        return;
    }

    tail->next = newNode;
    tail = newNode;
}

void insertPos(int pos, int val)
{
    if (pos == 1)
    {
        insertFront(val);
        return;
    }

    Node *newNode = new Node(val);

    Node *temp;
    temp = head;

    for (int i = 1; i < pos - 1; i++)
    {
        if (temp == nullptr)
        {
            cout << "\nInvalid position. ";
            delete newNode;
            return;
        }
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "\nInvalid position. ";
        delete newNode;
        return;
    }

    newNode->next = temp->next;
    temp->next = newNode;
```

DSA LAB 04 TASKS

```
    if (newNode->next == nullptr)
    {
        tail = newNode;
    }
}

void removeFront()
{
    if (head == nullptr)
    {
        return;
    }

    Node *temp;
    temp = head;
    head = head->next;
    delete temp;

    if (head == nullptr)
    {
        tail = nullptr;
    }
}

void removeBack()
{
    if (head == nullptr)
    {
        return;
    }

    if (head->next == nullptr)
    {
        delete head;
        head = nullptr;
        return;
    }

    Node *temp = head;
    while (temp->next != tail)
    {
```

DSA LAB 04 TASKS

```
        temp = temp->next;
    }

    delete temp->next;
    temp->next = nullptr;
    tail = temp;
}

void deletePos(int pos)
{
    if (head == nullptr)
    {
        cout << "\nList is empty. ";
        return;
    }

    if (pos == 1)
    {
        removeFront();
        return;
    }

    Node *temp;
    temp = head;

    for (int i = 1; i < pos - 1; i++)
    {
        if (temp->next == nullptr)
        {
            cout << "\nInvalid position. ";
            return;
        }

        temp = temp->next;
    }

    Node *deletenode = temp->next;

    if (deletenode == nullptr)
    {
        cout << "\nInvalid position.";
        return;
```

DSA LAB 04 TASKS

```
    }

    if (deletenode == tail)
    {
        tail = temp;
    }

    temp->next = deletenode->next;

    delete deletenode;
}

void display()
{
    Node *temp = head;

    cout << "\nList: ";

    while (temp != nullptr)
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }

    cout << "NULL";

    if (head != nullptr)
    {
        cout << "\nHead = " << head->data;
        cout << "\nTail = " << tail->data;
    }
    else
    {
        cout << "\nHead = NULL";
        cout << "\nTail = NULL";
    }

    cout << endl;
}

void modify()
{
```

DSA LAB 04 TASKS

```
Node *evenHead;
Node *evenTail;

Node *oddHead;
Node *oddTail;

evenHead = evenTail = oddHead = oddTail = nullptr;

Node *temp = head;

while (temp != nullptr)
{
   if ((temp->data % 2) == 0)
   {
      if (evenHead == nullptr)
      {
         evenHead = temp;
         evenTail = temp;
      }
      else
      {
         evenTail->next = temp;
         evenTail = temp;
      }

      temp = temp->next;
   }

   else
   {
      if (oddHead == nullptr)
      {
         oddHead = temp;
         oddTail = temp;
      }

      else
      {
         oddTail->next = temp;
         oddTail = temp;
      }
```

DSA LAB 04 TASKS

```
            temp = temp->next;
        }
    }

    if (evenHead != nullptr)
    {
        evenTail->next = oddHead;
        head = evenHead;
    }
    else
    {
        head = oddHead;
    }

    if (oddTail != nullptr)
    {
        oddTail->next = nullptr;
        tail = oddTail;
    }
    else
    {
        evenTail->next = nullptr;
        tail = evenTail;
    }
  }
};

int main()
{
  SLL list;
  int n;

  cout << "\nEnter number of values to be stored in linked list: ";
  cin >> n;

  for (int i = 1; i <= n; i++)
  {
    int val, choice;

    cout << "\nEnter value " << i << ": ";
    cin >> val;
```

DSA LAB 04 TASKS

```
    cout << "1. Insert at Front";
    cout << "\n2. Insert at Back";
    cout << "\nEnter choice: ";
    cin >> choice;

    if (choice == 1)
    {
        list.insertFront(val);
    }
    else if (choice == 2)
    {
        list.insertBack(val);
    }
    else
    {
        cout << "Invalid choice. Value not inserted.\n";
        i--;
    }
}

cout << "\n\n===== INITIAL LIST =====";
list.display();

int choice;

do
{
    cout << "\n\n===== MENU =====";
    cout << "\n1. Remove Front";
    cout << "\n2. Remove Back";
    cout << "\n3. Insert at Position";
    cout << "\n4. Delete at Position";
    cout << "\n5. Modify List";
    cout << "\n6. Display List";
    cout << "\n0. Exit";

    cout << "\n\nEnter choice: ";
    cin >> choice;

    if (choice == 1)
    {
        list.removeFront();
```

DSA LAB 04 TASKS

```
    cout << "\nAfter removing front:";
    list.display();
}

else if (choice == 2)
{
    list.removeBack();
    cout << "\nAfter removing back:";
    list.display();
}

else if (choice == 3)
{
    int pos, val;

    cout << "\nEnter position: ";
    cin >> pos;

    cout << "Enter value: ";
    cin >> val;

    list.insertPos(pos, val);

    cout << "\nAfter insertion:";
    list.display();
}

else if (choice == 4)
{
    int pos;

    cout << "\nEnter position to delete: ";
    cin >> pos;

    list.deletePos(pos);

    cout << "\nAfter deletion:";
    list.display();
}

else if(choice == 5)
{
```

DSA LAB 04 TASKS

```
            list.modify();
        }

        else if (choice == 6)
        {
            list.display();
        }

        else if (choice == 0)
        {
            cout << "\nProgram ended.";
        }

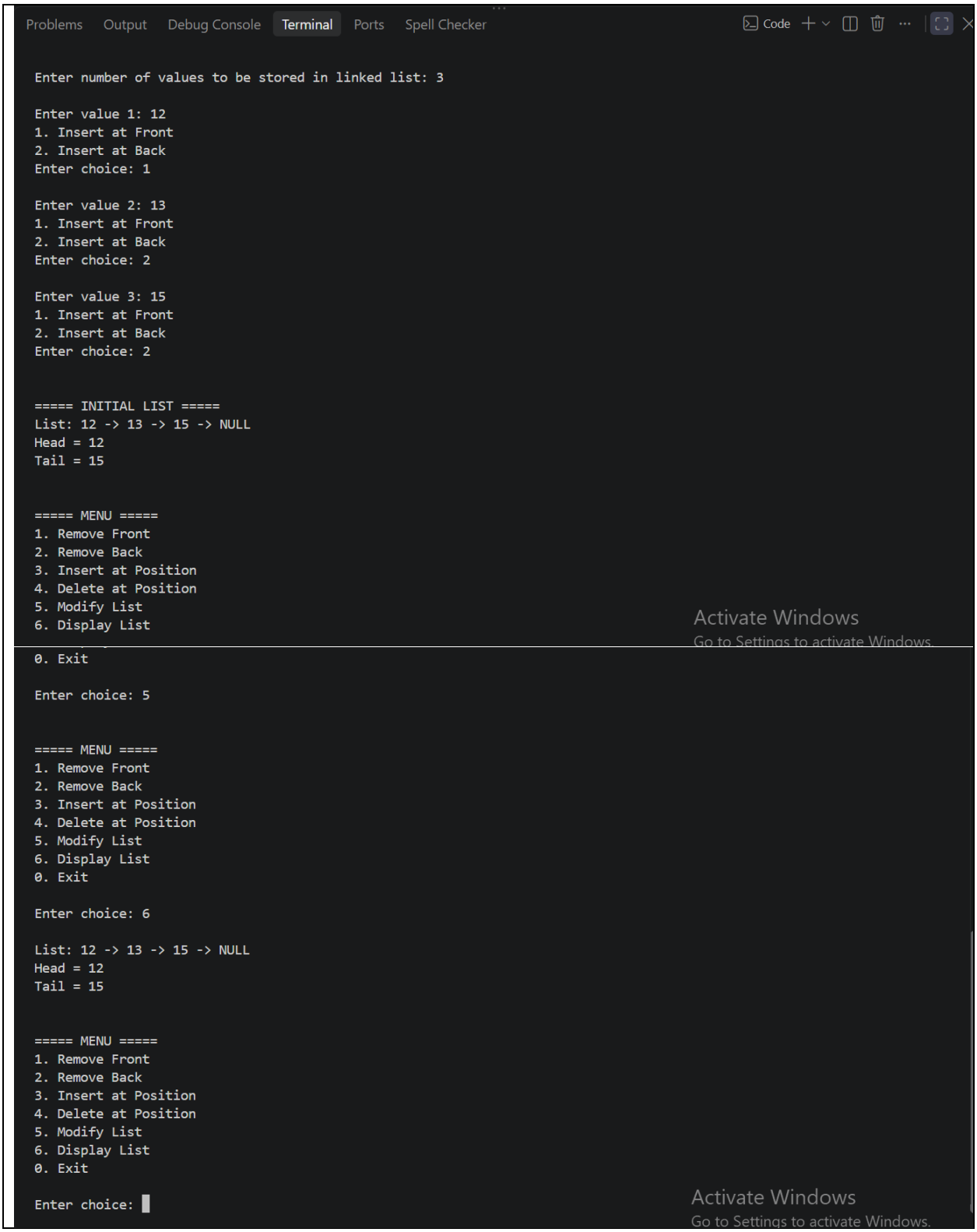
        else
        {
            cout << "\nInvalid choice.";
        }

    } while (choice != 0);

    return 0;
}
```

K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 04 TASKS



K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 04 TASKS

TASK 04:

```
#include <iostream>
using namespace std;

class Node
{
public:
  int data;
  Node *next;
  Node *prev;

  Node(int val)
  {
    data = val;
    next = prev = nullptr;
  }
};

class DLL
{
  Node *head;
  Node *tail;

public:
  DLL()
  {
    head = tail = nullptr;
  }

  void insert(int val)
  {
    Node *newNode = new Node(val);

    if (head == nullptr)
    {
      head = tail = newNode;

      return;
    }

    tail->next = newNode;
    newNode->prev = tail;
```

DSA LAB 04 TASKS

```
    tail = newNode;
}

void deletePos(int val)
{
    if (head == nullptr)
    {
        cout << "\nList is empty. ";
        return;
    }

    Node *temp = head;

    // Find the node
    while (temp != nullptr && temp->data != val)
    {
        temp = temp->next;
    }

    // Value not found
    if (temp == nullptr)
    {
        cout << "\nValue not found.";
        return;
    }

    // Only one node
    if (head == tail)
    {
        head = tail = nullptr;
    }

    // Delete head
    else if (temp == head)
    {
        head = head->next;
        head->prev = nullptr;
    }

    // Delete tail
    else if (temp == tail)
    {
```

DSA LAB 04 TASKS

```
        tail = tail->prev;
        tail->next = nullptr;
    }

    // Delete middle node
    else
    {
        temp->prev->next = temp->next;
        temp->next->prev = temp->prev;
    }

    delete temp;
}

void displayForward()
{
    Node *temp = head;

    cout << endl;

    while (temp != nullptr)
    {
        cout << temp->data << "->";
        temp = temp->next;
    }
    cout<<"NULL";
}

void displayBackword()
{
    Node *temp = tail;

    cout << endl;

    cout<<"NULL<-";
    while (temp != nullptr)
    {
        cout << temp->data << "<-";
        temp = temp->prev;
    }
}
};
```

K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 04 TASKS

```
int main()
{
   DLL list;

   int n;
   cout << "Enter number of nodes: ";
   cin >> n;

   int val;

   for (int i = 1; i <= n; i++)
   {
      cout << "Enter value " << i << ": ";
      cin >> val;

      list.insert(val);
   }

   cout << "\nOriginal list (Forward): ";
   list.displayForward();

   cout << "\nOriginal list (Backward): ";
   list.displayBackword();

   cout << "\n\nEnter value to delete: ";
   cin >> val;

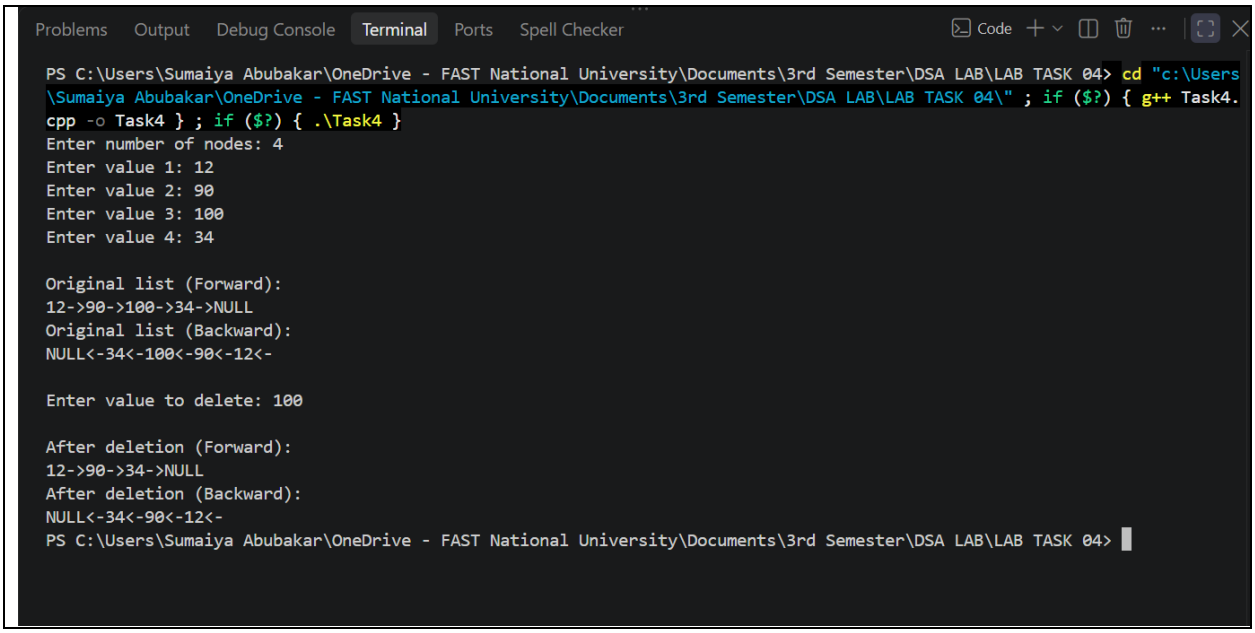
   list.deletePos(val);

   cout << "\nAfter deletion (Forward): ";
   list.displayForward();

   cout << "\nAfter deletion (Backward): ";
   list.displayBackword();

   return 0;
}
```

DSA LAB 04 TASKS



TASK 05:

```
#include <iostream>
using namespace std;

class Node
{
public:
  int data;
  Node *next;
  Node *prev;

  Node(int val)
  {
    data = val;
    next = prev = nullptr;
  }
};

class CDLL
{
  Node *head;
  Node *tail;

public:
```

K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 04 TASKS

```
CDLL()
{
   head = tail = nullptr;
}

void insertAtHead(int val)
{
   Node *newNode = new Node(val);

   if (head == nullptr)
   {
      head = tail = newNode;
      newNode->next = head;
      newNode->prev = tail;
      return;
   }

   newNode->next = head;
   head->prev = newNode;

   head = newNode;
   tail->next = head;
   head->prev = tail;
}

void insertAtTail(int val)
{
   Node *newNode = new Node(val);

   if (head == nullptr)
   {
      head = tail = newNode;
      newNode->next = head;
      newNode->prev = tail;
      return;
   }

   tail->next = newNode;
   newNode->prev = tail;
   tail = newNode;
   tail->next = head;
   head->prev = tail;
```

DSA LAB 04 TASKS

```
}

void moveForward(int pos)
{
   if (head == nullptr)
   {
      cout << "\nList is empty. ";
      return;
   }

   if (pos < 1 || pos > getCount())
   {
      cout << "\nInvalid node. ";
      return;
   }

   Node *temp = head;
   for (int i = 1; i < pos; i++)
   {
      temp = temp->next;
   }

   cout << "\nNode at position " << pos << " : " << temp->data << endl;
}

int getCount()
{
   int count = 0;
   Node *temp = head;

   if (head == nullptr)
   {
      return 0;
   }

   do
   {
      count++;
      temp = temp->next;
   } while (temp != head);

   return count;
```

DSA LAB 04 TASKS

```
}
void moveBackward(int pos)
{
  if (head == nullptr)
  {
    cout << "\nList is empty. ";
    return;
  }

  if (pos < 1 || pos > getCount())
  {
    cout << "\nInvalid node. ";
    return;
  }

  Node *temp = tail;
  for (int i = getCount(); i > pos; i--)
  {
    temp = temp->prev;
  }

  cout << "\nNode at position " << pos << " : " << temp->data << endl;
}

void display()
{
  Node *temp = head;
  cout << "\n";

  if (head == nullptr)
  {
    cout << "\nList is empty.";
    return;
  }
  do
  {
    cout << temp->data << " -> ";
    temp = temp->next;
  }

  while (temp != head);
}
```

DSA LAB 04 TASKS

```
};

int main()
{
  CDLL list;

  int n;
  cout << "Enter number of nodes: ";
  cin >> n;

  int val;

  for (int i = 1; i <= n; i++)
  {
    int val, choice;

    cout << "\nEnter value " << i << ": ";
    cin >> val;

    cout << "1. Insert at Front";
    cout << "\n2. Insert at Back";
    cout << "\nEnter choice: ";
    cin >> choice;

    if (choice == 1)
    {
      list.insertAtHead(val);
    }
    else if (choice == 2)
    {
      list.insertAtTail(val);
    }
    else
    {
      cout << "Invalid choice. Value not inserted.\n";
      i--;
    }
  }

  cout << "\n\n===== INITIAL LIST =====";
  list.display();
```

DSA LAB 04 TASKS

```
int choice;
int pos;

do
{
   cout << "\n\n===== MENU =====";
   cout << "\n1. Move forward";
   cout << "\n2. Move backward";
   cout << "\n0. Exit";

   cout << "\n\nEnter choice: ";
   cin >> choice;

   if (choice == 1)
   {
      cout << "\nEnter position : ";
      cin >> pos;
      list.moveForward(pos);
      cout << "\nAfter moving forward:";
      list.display();
   }

   else if (choice == 2)
   {
      cout << "\nEnter position : ";
      cin >> pos;
      list.moveBackward(pos);
      cout << "\nAfter moving back:";
      list.display();
   }

   else if (choice == 0)
   {
      cout << "\nProgram ended.";
   }

   else
   {
      cout << "\nInvalid choice.";
   }

} while (choice != 0);
```

DSA LAB 04 TASKS

```
}
```

Problems Output Debug Console Terminal Ports Spell Checker Code

```
Enter number of nodes: 4

Enter value 1: 1
1. Insert at Front
2. Insert at Back
Enter choice: 1

Enter value 2: 2
1. Insert at Front
2. Insert at Back
Enter choice: 2

Enter value 3: 3
1. Insert at Front
2. Insert at Back
Enter choice: 2

Enter value 4: 4
1. Insert at Front
2. Insert at Back
Enter choice: 2


===== INITIAL LIST =====
1 -> 2 -> 3 -> 4 ->

===== MENU =====
1. Move forward
2. Move backward
0. Exit

Enter choice: 2
Enter position : 2

Node at position 2 : 2

After moving back:
1 -> 2 -> 3 -> 4 ->

===== MENU =====
1. Move forward
2. Move backward
0. Exit

Enter choice: 1

Enter position : 4

Node at position 4 : 4

After moving forward:
1 -> 2 -> 3 -> 4 ->

===== MENU =====
1. Move forward
2. Move backward
0. Exit

Enter choice:
```

Activate Windows
Activate Windows
Go to Settings to activate Windows.

TASK 06:

K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 04 TASKS

```
#include <iostream>
using namespace std;

class Node
{
public:
   string teamName;
   int points;
   Node *next;

   Node(string name, int pts)
   {
      teamName = name;
      points = pts;
      next = nullptr;
   }
};

class CSL
{
   Node *head;
   Node *tail;

public:
   CSL()
   {
      head = tail = nullptr;
   }

   bool search(string name)
   {
      if (head == nullptr)
      {
         return false;
      }

      Node *temp = head;

      do
      {
         if (temp->teamName == name)
         {
```

DSA LAB 04 TASKS

```
            return true;
        }

        temp = temp->next;
    }

    while (temp != head);

    return false;
}

void addTeam(string name, int pts)
{
    if (pts < 0 || pts > 150)
    {
        cout << "Invalid points. Points must be between 0 and 150.\n";
        return;
    }

    if (search(name))
    {
        cout << "Duplicate team name. Team not added.\n";
        return;
    }

    Node *newNode = new Node(name, pts);

    if (head == nullptr)
    {
        head = tail = newNode;
        newNode->next = head;
        return;
    }

    tail->next = newNode;
    tail = newNode;
    tail->next = head;
}

void display()
{
    if (head == nullptr)
```

DSA LAB 04 TASKS

```
    {
        cout << "List is empty.\n";
        return;
    }

    Node *temp = head;

    do
    {
        cout << temp->teamName << " - Points: " << temp->points << endl;

        temp = temp->next;

    } while (temp != head);
}

void removeTeam(string name)
{
    if (head == nullptr)
    {
        cout << "\nList is empty.";
        return;
    }

    Node *temp = head;
    Node *prev = tail;

    do
    {
        if (temp->teamName == name)
        {
            if (temp == head && temp == tail)
            {
                delete temp;
                head = tail = nullptr;
                cout << "\nTeam removed.\n";
                return;
            }

            if (temp == head)
            {
                head = head->next;
```

DSA LAB 04 TASKS

```
            tail->next = head;

            delete temp;

            cout << "\nTeam removed.\n";
            return;
        }

        prev->next = temp->next;

        if (temp == tail)
        {
            tail = prev;
            tail->next = head;
        }

        delete temp;

        cout << "Team removed.\n";
        return;
      }

      prev = temp;
      temp = temp->next;
   }

   while (temp != head);

   cout << "\nTeam not found.";
}

Node *findHighestScoreTeam()
{
   if (head == nullptr)
   {
      return nullptr;
   }

   Node *highest = head;
   Node *temp = head->next;

   while (temp != head)
```

DSA LAB 04 TASKS

```
    {
        if (temp->points > highest->points)
        {
            highest = temp;
        }

        temp = temp->next;
    }

    return highest;
}

Node *findLowestScoreTeam()
{
    if (head == nullptr)
    {
        return nullptr;
    }

    Node *Lowest = head;
    Node *temp = head->next;

    while (temp != head)
    {
        if (temp->points < Lowest->points)
        {
            Lowest = temp;
        }

        temp = temp->next;
    }

    return Lowest;
}

void updatePoints()
{
    if (head == nullptr)
    {
        return;
    }
```

DSA LAB 04 TASKS

```
    Node *temp = head;

    int position = 1;

    do
    {
        if (position % 2 == 1)
        {
            temp->points = temp->points - 20;
        }
        else
        {
            temp->points = temp->points + 10;
        }

        if (temp->points < 0)
        {
            temp->points = 0;
        }

        if (temp->points > 150)
        {
            temp->points = 150;
        }

        temp = temp->next;
        position++;
    }

    while (temp != head);
}

Node *findLowestFrom(Node *start)
{
    Node *lowest = start;
    Node *temp = start->next;

    do
    {
        if (temp->points < lowest->points)
        {
            lowest = temp;
```

DSA LAB 04 TASKS

```
        }

        temp = temp->next;

    }

    while (temp != start);

    return lowest;
}

void knockOutChallenge()
{
    if (head == nullptr)
    {
        cout << "No teams available.\n";
        return;
    }

    if (head == tail)
    {
        cout << "Only one team exists.\n";
        cout << "Champion: " << head->teamName << endl;
        return;
    }

    string startTeam;
    int count;

    cout << "Enter starting team: ";
    cin >> startTeam;

    cout << "Enter counting number: ";
    cin >> count;

    if (count <= 0)
    {
        cout << "Counting number must be greater than 0.\n";
        return;
    }

    Node *current = head;
```

DSA LAB 04 TASKS

```
bool found = false;

do
{
   if (current->teamName == startTeam)
   {
      found = true;
      break;
   }

   current = current->next;

} while (current != head);

if (!found)
{
   cout << "Starting team not found.\n";
   return;
}

while (head != tail)
{
   Node *selected = current;

   for (int i = 1; i < count; i++)
   {
      selected = selected->next;
   }

   Node *prev = selected;

   while (prev->next != selected)
   {
      prev = prev->next;
   }

   Node *highest = findHighestScoreTeam();

   Node *eliminated;
   Node *eliminatedPrev;

   if (selected->points == highest->points)
```

DSA LAB 04 TASKS

```
{
   cout << "Selected team " << selected->teamName
      << " cannot be eliminated because it has the highest points.\n";
   eliminated = findLowestFrom(current);

   eliminatedPrev = eliminated;

   while (eliminatedPrev->next != eliminated)
   {
      eliminatedPrev = eliminatedPrev->next;
   }
}
else
{
   eliminated = selected;
   eliminatedPrev = prev;
}

Node *nextStart = eliminated->next;

eliminatedPrev->next = eliminated->next;

if (eliminated == head)
{
   head = eliminated->next;
}

if (eliminated == tail)
{
   tail = eliminatedPrev;
}

tail->next = head;

cout << "Eliminated team: " << eliminated->teamName << endl;

delete eliminated;

current = nextStart;

updatePoints();
```

DSA LAB 04 TASKS

```
        display();
    }

    cout << "\nFIFA Knockout Challenge Champion: " << head->teamName << endl;
  }
};

int main()
{
  CSL list;

  int choice;

  do
  {
    cout << "\n===== FIFA WORLD CUP KNOCKOUT CHALLENGE =====\n";
    cout << "1. Add Team\n";
    cout << "2. Remove Team\n";
    cout << "3. Search Team\n";
    cout << "4. Display Teams\n";
    cout << "5. Start Knockout Challenge\n";
    cout << "6. Exit\n";

    cout << "Enter your choice: ";
    cin >> choice;

    if (choice == 1)
    {
      string name;
      int points;

      cout << "Enter team name: ";
      cin >> name;

      cout << "Enter points: ";
      cin >> points;

      list.addTeam(name, points);
    }

    else if (choice == 2)
    {
```

DSA LAB 04 TASKS

```
    string name;

    cout << "Enter team name to remove: ";
    cin >> name;

    list.removeTeam(name);
  }

  else if (choice == 3)
  {
    string name;

    cout << "Enter team name to search: ";
    cin >> name;

    if (list.search(name))
    {
      cout << "Team found.\n";
    }
    else
    {
      cout << "Team not found.\n";
    }
  }

  else if (choice == 4)
  {
    list.display();
  }

  else if (choice == 5)
  {
    list.knockOutChallenge();
  }

  else if (choice == 6)
  {
    cout << "Program ended.\n";
  }

  else
  {
```

DSA LAB 04 TASKS

```
        cout << "Invalid choice.\n";
    }

  } while (choice != 6);

  return 0;
}
```

```
Problems  Output  Debug Console  Terminal  Ports  Spell Checker

===== FIFA WORLD CUP KNOCKOUT CHALLENGE =====
1. Add Team
2. Remove Team
3. Search Team
4. Display Teams
5. Start Knockout Challenge
6. Exit
Enter your choice: 1
Enter team name: Germany
Enter points: 100

===== FIFA WORLD CUP KNOCKOUT CHALLENGE =====
1. Add Team
2. Remove Team
3. Search Team
4. Display Teams
5. Start Knockout Challenge
6. Exit
Enter your choice: 1
Enter team name: France
Enter points: 50

===== FIFA WORLD CUP KNOCKOUT CHALLENGE =====
1. Add Team
2. Remove Team
3. Search Team
4. Display Teams
5. Start Knockout Challenge
6. Exit
Enter your choice: 1
Enter team name: Brazil
```

Activate Windows

K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 04 TASKS

```
Enter points: 78

===== FIFA WORLD CUP KNOCKOUT CHALLENGE =====
1. Add Team
2. Remove Team
3. Search Team
4. Display Teams
5. Start Knockout Challenge
6. Exit
Enter your choice: 4
Germany - Points: 100
France - Points: 50
Brazil - Points: 78

===== FIFA WORLD CUP KNOCKOUT CHALLENGE =====
1. Add Team
2. Remove Team
3. Search Team
4. Display Teams
5. Start Knockout Challenge
6. Exit
Enter your choice: 5
Enter starting team: Germany
Enter counting number: 2
Eliminated team: France
Germany - Points: 80
Brazil - Points: 88
Eliminated team: Germany
Brazil - Points: 68

FIFA Knockout Challenge Champion: Brazil
```

Activate Windows

TASK 07:

```
#include <iostream>
using namespace std;

class Node
{
public:
   string teamName;
   int points;
   Node *next;
   Node *prev;

   Node(string name, int pts)
   {
      teamName = name;
      points = pts;
      next = nullptr;
      prev = nullptr;
   }
};
```

K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 04 TASKS

```
class CSL
{
   Node *head;
   Node *tail;

public:
   CSL()
   {
      head = tail = nullptr;
   }

   bool search(string name)
   {
      if (head == nullptr)
      {
         return false;
      }

      Node *temp = head;

      do
      {
         if (temp->teamName == name)
         {
            return true;
         }

         temp = temp->next;
      }

      while (temp != head);

      return false;
   }

   void addTeam(string name, int pts)
   {
      if (pts < 0 || pts > 150)
      {
         cout << "Invalid points. Points must be between 0 and 150.\n";
         return;
```

DSA LAB 04 TASKS

```
    }

    if (search(name))
    {
        cout << "Duplicate team name. Team not added.\n";
        return;
    }

    Node *newNode = new Node(name, pts);

    if (head == nullptr)
    {
        head = tail = newNode;
        newNode->next = head;
        newNode->prev = tail;
        return;
    }

    tail->next = newNode;
    newNode->prev = tail;
    tail = newNode;
    tail->next = head;
    head->prev = tail;
}

void display()
{
    if (head == nullptr)
    {
        cout << "List is empty.\n";
        return;
    }

    Node *temp = head;

    do
    {
        cout << temp->teamName << " - Points: " << temp->points << endl;

        temp = temp->next;

    } while (temp != head);
```

DSA LAB 04 TASKS

```
}

void removeTeam(string name)
{
   if (head == nullptr)
   {
      cout << "\nList is empty.";
      return;
   }

   Node *temp = head;

   do
   {
      if (temp->teamName == name)
      {
         if (temp == head && temp == tail)
         {
            delete temp;
            head = tail = nullptr;
            cout << "\nTeam removed.\n";
            return;
         }

         temp->prev->next = temp->next;
         temp->next->prev = temp->prev;

         if (temp == head)
         {
            head = temp->next;
         }

         if (temp == tail)
         {
            tail = temp->prev;
         }

         tail->next = head;
         head->prev = tail;

         delete temp;
```

DSA LAB 04 TASKS

```
            cout << "Team removed.\n";
            return;
        }

        temp = temp->next;
    }

    while (temp != head);

    cout << "\nTeam not found.";
}

Node *findHighestScoreTeam()
{
    if (head == nullptr)
    {
        return nullptr;
    }

    Node *highest = head;
    Node *temp = head->next;

    while (temp != head)
    {
        if (temp->points > highest->points)
        {
            highest = temp;
        }

        temp = temp->next;
    }

    return highest;
}

Node *findLowestScoreTeam()
{
    if (head == nullptr)
    {
        return nullptr;
    }
```

DSA LAB 04 TASKS

```
    Node *Lowest = head;
    Node *temp = head->next;

    while (temp != head)
    {
        if (temp->points < Lowest->points)
        {
            Lowest = temp;
        }

        temp = temp->next;
    }

    return Lowest;
}

void updatePoints()
{
    if (head == nullptr)
    {
        return;
    }

    Node *temp = head;

    int position = 1;

    do
    {
        if (position % 2 == 1)
        {
            temp->points = temp->points - 20;
        }
        else
        {
            temp->points = temp->points + 10;
        }

        if (temp->points < 0)
        {
            temp->points = 0;
        }
```

DSA LAB 04 TASKS

```
        if (temp->points > 150)
        {
            temp->points = 150;
        }

        temp = temp->next;
        position++;
    }

    while (temp != head);
}

// Node *findLowestFrom(Node *start)
// {
//     Node *lowest = start;
//     Node *temp = start->next;

//     do
//     {
//         if (temp->points < lowest->points)
//         {
//             lowest = temp;
//         }

//         temp = temp->next;

//     }

//     while (temp != start);

//     return lowest;
// }

void knockOutChallenge()
{
    if (head == nullptr)
    {
        cout << "No teams available.\n";
        return;
    }
```

DSA LAB 04 TASKS

```
if (head == tail)
{
   cout << "Only one team exists.\n";
   cout << "Champion: " << head->teamName << endl;
   return;
}

string startTeam;
int count;

cout << "Enter starting team: ";
cin >> startTeam;

cout << "Enter counting number: ";
cin >> count;

if (count <= 0)
{
   cout << "Counting number must be greater than 0.\n";
   return;
}

Node *current = head;
bool found = false;

do
{
   if (current->teamName == startTeam)
   {
      found = true;
      break;
   }

   current = current->next;

} while (current != head);

if (!found)
{
   cout << "Starting team not found.\n";
   return;
}
```

DSA LAB 04 TASKS

```
bool forward = true;

while (head != tail)
{
   Node *selected = current;

   for (int i = 1; i < count; i++)
   {
      if (forward)
      {
         selected = selected->next;
      }
      else
      {
         selected = selected->prev;
      }
   }

   Node *highest = findHighestScoreTeam();

   Node *eliminated;

   bool protectionTriggered = false;

   if (selected->points == highest->points)
   {
      cout << "Selected team " << selected->teamName
           << " cannot be eliminated because it has the highest points.\n";
      eliminated = findLowestScoreTeam();
      protectionTriggered = true;
   }
   else
   {
      eliminated = selected;
   }

   Node *nextForward = eliminated->next;
   Node *nextBackward = eliminated->prev;

   eliminated->prev->next = eliminated->next;
   eliminated->next->prev = eliminated->prev;
```

DSA LAB 04 TASKS

```
    // Node *nextStart = eliminated->next;

    // eliminatedPrev->next = eliminated->next;

    if (eliminated == head)
    {
        head = eliminated->next;
    }

    if (eliminated == tail)
    {
        tail = eliminated->prev;
    }

    tail->next = head;
    head->prev = tail;

    cout << "Eliminated team: " << eliminated->teamName << endl;

    delete eliminated;

    if (protectionTriggered)
    {
        forward = !forward;
    }

    if (forward)
    {
        current = nextForward;
    }
    else
    {
        current = nextBackward;
    }

    // current = nextStart;

    updatePoints();

    display();
}
```

DSA LAB 04 TASKS

```
    cout << "\nFIFA Knockout Challenge Champion: " << head->teamName << endl;
  }
};

int main()
{
  CSL list;

  int choice;

  do
  {
    cout << "\n===== FIFA WORLD CUP KNOCKOUT CHALLENGE =====\n";
    cout << "1. Add Team\n";
    cout << "2. Remove Team\n";
    cout << "3. Search Team\n";
    cout << "4. Display Teams\n";
    cout << "5. Start Knockout Challenge\n";
    cout << "6. Exit\n";

    cout << "Enter your choice: ";
    cin >> choice;

    if (choice == 1)
    {
      string name;
      int points;

      cout << "Enter team name: ";
      cin >> name;

      cout << "Enter points: ";
      cin >> points;

      list.addTeam(name, points);
    }

    else if (choice == 2)
    {
      string name;
```

DSA LAB 04 TASKS

```
    cout << "Enter team name to remove: ";
    cin >> name;

    list.removeTeam(name);
  }

  else if (choice == 3)
  {
    string name;

    cout << "Enter team name to search: ";
    cin >> name;

    if (list.search(name))
    {
      cout << "Team found.\n";
    }
    else
    {
      cout << "Team not found.\n";
    }
  }

  else if (choice == 4)
  {
    list.display();
  }

  else if (choice == 5)
  {
    list.knockOutChallenge();
  }

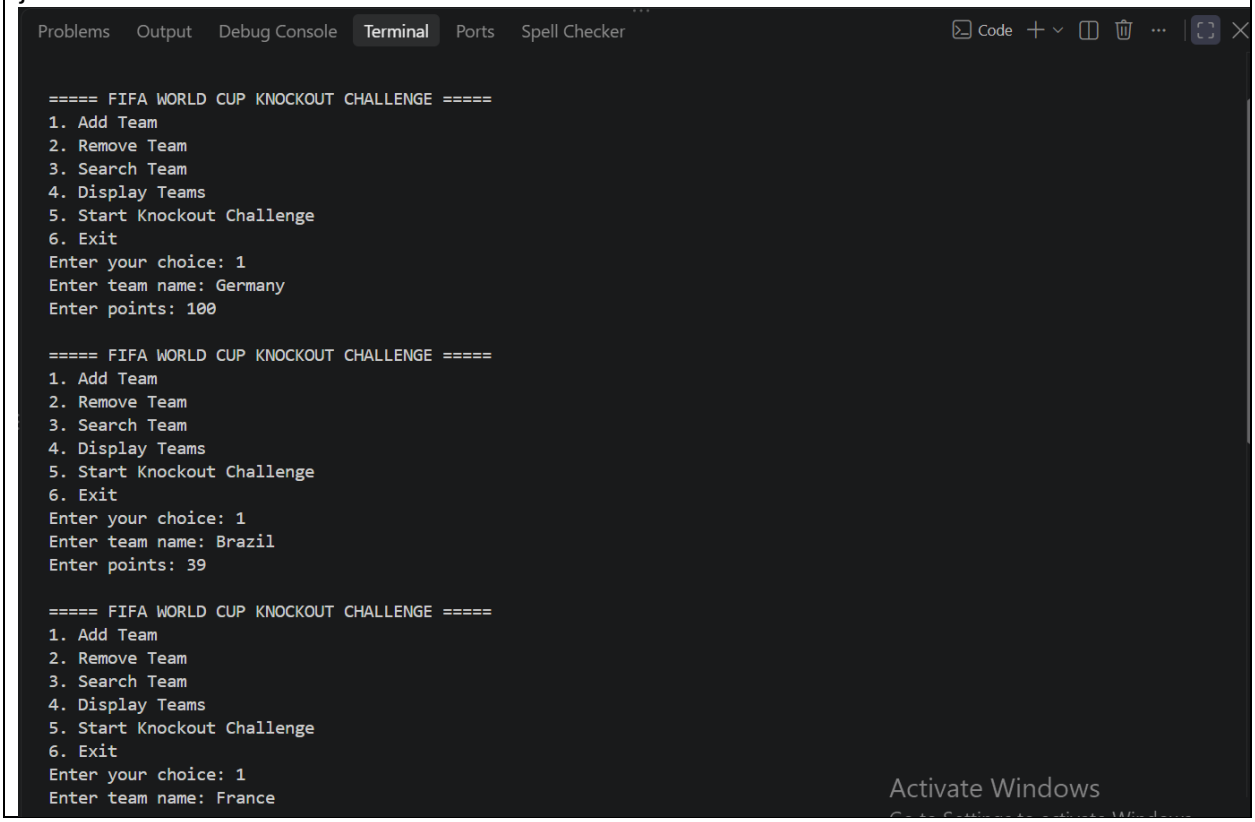
  else if (choice == 6)
  {
    cout << "Program ended.\n";
  }

  else
  {
    cout << "Invalid choice.\n";
  }
```

DSA LAB 04 TASKS

```
    } while (choice != 6);

    return 0;
}
```



K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 04 TASKS

```
Enter points: 145

===== FIFA WORLD CUP KNOCKOUT CHALLENGE =====
1. Add Team
2. Remove Team
3. Search Team
4. Display Teams
5. Start Knockout Challenge
6. Exit
Enter your choice: 5
Enter starting team: Brazil
Enter counting number: 1
Eliminated team: Brazil
Germany - Points: 80
France - Points: 150
Selected team France cannot be eliminated because it has the highest points.
Eliminated team: Germany
France - Points: 130

FIFA Knockout Challenge Champion: France

===== FIFA WORLD CUP KNOCKOUT CHALLENGE =====
1. Add Team
2. Remove Team
3. Search Team
4. Display Teams
5. Start Knockout Challenge
6. Exit
Enter your choice:
```